



Pipelining & Hazards in Processors

PPT PRESENTED BY

NAME : Srinivas Rao Tammireddy

ROLL NO : 257Y1D5805

YEAR : I Year II Semester

ACADEMIC YEAR : 2025 - 2026

What is Pipelining?

Pipelining is an implementation technique where multiple instructions are overlapped in execution, similar to a manufacturing assembly line. It maximizes processor instruction throughput.

Core Pipeline Architecture

- **Stage Division:** Instructions are split into sequential steps, each executed by a dedicated hardware stage.
- **Overlapped Execution:** When stage 1 finishes instruction A and moves it to stage 2, stage 1 immediately begins instruction B.
- **Ideal Speedup:** A pipeline of k stages can achieve a speedup factor of k in the ideal case (completing 1 instruction per clock cycle).
- **Latency vs. Throughput:** Pipelining does not decrease the execution latency of a single instruction. It increases the throughput of completed instructions.

Standard 5-Stage MIPS Pipeline

Most RISC designs use these classic 5 stages:

1. **Instruction Fetch (IF):** Load instruction from memory.
2. **Instruction Decode (ID):** Translate opcode and read register values.
3. **Execute (EX):** Run arithmetic operations in the ALU.
4. **Memory Access (MEM):** Read or write data from cache memory.
5. **Write-Back (WB):** Write results back into register files.

Pipeline Hazards

Hazards are situations that prevent the next instruction in the instruction stream from executing in its designated clock cycle:

1. Structural Hazards

- Conflict: Hardware cannot support all stage combinations in the same clock cycle.
- Cause: E.g., single memory port for both fetching instructions (IF) and loading data (MEM).
- Solution: Separate instruction and data caches (Harvard Architecture).

2. Data Hazards

- Conflict: An instruction depends on the result of a prior instruction still in the pipeline.
- Cause: Read-After-Write (RAW) dependencies.
- Solution: Data forwarding (ALU bypass), compiler instruction reordering, or inserting bubbles.

3. Control Hazards

- Conflict: Imbalances caused by branch or jump instructions, which change the instruction flow.
- Cause: Target address is unknown during Fetch stage.
- Solution: Branch prediction hardware, branch delay slots.

Resolving Data Hazards

How CPU designers resolve data dependencies in high-performance pipelines:

Data Forwarding & Bypassing

- **The Concept:** Route the ALU calculation result directly to the ALU input stages of subsequent instructions, skipping the Write-Back register step.
- **Dynamic Routing:** Bypass multiplexers feed the EX output directly back to the EX input, resolving RAW hazards in one cycle.
- **Load-Use Stalls:** If an instruction depends on a memory LOAD (MEM stage), forwarding is not enough because data is not ready until the end of MEM. The pipeline MUST insert a 1-cycle stall (bubble).

Compiler Solutions & Bubbles

- **Pipeline Stalling (Bubbles):** Hardware detects dependencies and pauses the fetch/decode stages, inserting a 'no-operation' (NOP) instruction bubble to let prior instructions finish.
- **Compiler Reordering:** Optimizing compilers analyze dependencies and insert independent instructions into hazard gaps, avoiding hardware stalls.
- **Branch Delay Slot:** Compilers place an independent instruction after a branch, executing it regardless of the branch outcome, avoiding control stalls.

Applications of Pipelining

Pipelining is a fundamental design principle across modern processor cores:



CISC & RISC Processors

Every modern desktop CPU (Intel x86, AMD, ARM) utilizes deep pipelines (typically 10-20 stages) to achieve gigahertz clock speeds and high instruction throughput.



GPU Vector Cores

Graphic processors use deep, specialized pipeline channels to run vector math on millions of pixels concurrently, accelerating graphics rendering.



Network Router Switches

Router switches apply pipelining to packet header checking and route lookup, forwarding network packets with minimal latency.



Supercomputer Vector Units

HPC vector units use deep pipelining to compute floating-point arithmetic (matrix operations) rapidly, powering scientific modeling.



THANK YOU

Department of Computer Science and Engineering
Marri Laxman Reddy Institute of Technology and Management